

Costanti

Costanti

Come definire valori costanti in C++ con const e

Cos'è una costante?

Immagina di scrivere un programma che usa il numero pi greco (3.14159...) in dieci punti diversi. Se un giorno vuoi usare una versione più precisa, devi cambiarla in dieci posti.

Oppure potresti avere il numero **18** sparso nel codice. Chi legge non sa se è l'età minima, un limite di velocità o qualcos'altro.

Le **costanti** risolvono entrambi i problemi: dai un nome a un valore che non cambia mai, e lo scrivi in un unico posto.

La parola chiave **const**

Scrivi **const** prima del tipo per rendere una variabile costante. Una volta assegnato il valore, non può più essere cambiato.

```
const double PI = 3.14159265358979;  
const int ORE_PER_GIORNO = 24;  
const int ETA_MINIMA = 18;
```

Se provi a modificare una costante, il compilatore ti blocca con un errore:

```
const int MAX = 100;  
MAX = 200; // ERRORE: non puoi modificare una costante
```

Questo è un vantaggio: il compilatore ti protegge da modifiche accidentali.

La convenzione dei nomi

Per le costanti si usa la convenzione di scrivere **tutto in maiuscolo**, con le parole separate da underscore. Così si riconoscono a colpo d'occhio nel codice:

```
const double PI = 3.14159;
const int ETA_MINIMA = 18;
const int MAX_TENTATIVI = 3;
const double VELOCITA_LUCE = 299792458.0;
```

Un altro modo: #define

Prima che esistesse `const`, i programmatori usavano la direttiva `#define`:

```
#define PI 3.14159
#define MAX_STUDENTI 30
```

Con `#define`, il compilatore sostituisce ogni occorrenza di `PI` con `3.14159` prima di compilare il codice. Non è una vera variabile: è una semplice sostituzione di testo.

```
#include <iostream>
#define PI 3.14159
#define RAGGIO 5

using namespace std;

int main() {
    // Il compilatore sostituisce PI con 3.14159 e RAGGIO con 5
    double area = PI * RAGGIO * RAGGIO;
    cout << "Area: " << area << endl;
    return 0;
}
```

const o #define ? Usa const

Oggi si preferisce sempre `const`. Ecco perché:

Caratteristica	<code>const</code>	<code>#define</code>
Ha un tipo definito	Sì	No
Il compilatore la controlla	Sì	No
Visibile nel debugger	Sì	No
Rispetta i blocchi di codice	Sì	No

```
// Meglio: const ha tipo e viene controllata
const double PI = 3.14159;

// Meno sicuro: #define non ha tipo
#define PI 3.14159
```

constexpr : costanti calcolate durante la compilazione

In C++ moderno esiste anche `constexpr`. Si usa per costanti il cui valore può essere calcolato già durante la compilazione (prima ancora che il programma venga eseguito):

```
constexpr int SECONDI_PER_MINUTO = 60;
constexpr int MINUTI_PER_ORA = 60;
// Il compilatore calcola 60 * 60 = 3600 prima ancora di eseguire il
programma
constexpr int SECONDI_PER_ORA = SECONDI_PER_MINUTO * MINUTI_PER_ORA;
```

Per ora puoi usare tranquillamente `const`. Imparerai `constexpr` più avanti.

Esempio pratico

```
#include <iostream>
using namespace std;

// Costanti definite una volta sola, usate ovunque nel programma
const double PI = 3.14159265358979;
const double GRAVITA = 9.81; // accelerazione di gravità in m/s²
const int MAX_VOTO = 10;

int main() {
    // Calcolo dell'area di un cerchio con raggio 4
    double raggio = 4.0;
    double area = PI * raggio * raggio;
    cout << "Area cerchio (r=" << raggio << "): " << area << endl;

    // Calcolo dell'energia potenziale di un oggetto
    double massa = 70.0; // kg
    double altezza = 5.0; // m
    double energia = massa * GRAVITA * altezza;
    cout << "Energia potenziale: " << energia << " J" << endl;

    // Uso della costante del voto massimo
    int voto = 8;
    cout << "Voto: " << voto << "/" << MAX_VOTO << endl;

    return 0;
}
```

Output:

```
Area cerchio (r=4): 50.2655
Energia potenziale: 3434.5 J
Voto: 8/10
```